

PYTHON FORGE

Updated Comprehensive Review and Roadmap - Revision 1.1

Generated: 10 August 2026 | Scope: current workspace evidence and the next development backlog

This document updates Python_Forge_Comprehensive_Review_and_Roadmap.pdf. It marks work that is implemented and validated, separates partial or blocked work from future work, and records the development phases recommended from this point forward.

SOURCE LIMITATION

The original PDF is preserved unchanged at the repository root. Its readable metadata identifies the title Python Forge - Comprehensive Review & Bounded Roadmap and a 9 August 2026 creation date, but its body is stored in compressed streams that were not reliably extractable. This revision therefore uses repository source, tests, build output, and device evidence rather than claiming a line-by-line reproduction of unreadable original sections.

1. Executive status

The main application-quality recommendations have been implemented and validated locally. The project is not yet production-ready for hostile arbitrary-code execution or public release because the runtime is still in-process and the release artifact is unsigned.

[IMPLEMENTED] Core product hardening

Runtime request limits, best-effort execution deadlines, deterministic archive packaging, curriculum contracts, guarded navigation, progression reconciliation, achievement persistence, semantic controls, and responsive layout protections are present in the workspace.

[PARTIAL] Release and evidence gates

The release AAB build and native-symbol check pass locally, and a CI workflow exists, but hosted CI provenance, integration/device artifacts, signing verification, and full roadmap traceability remain incomplete.

[BLOCKED] Production sandbox and signing

A separate killable execution worker or external sandbox has not been implemented. Production signing secrets and a keystore are not available, so the locally generated release AAB is unsigned.

Status legend

[IMPLEMENTED] Completed in source and supported by validation evidence. [PARTIAL] Some implementation exists, but an acceptance gate or evidence is still missing. [PLANNED] New work proposed in this revision. [BLOCKED] Cannot be completed with the current architecture or unavailable credentials. [EVIDENCE GAP] Work may exist, but provenance or verification is not recorded.

2. Implemented changes to date

The following items are marked implemented because they are visible in the current codebase and were covered by the latest local validation pass.

2.1 Runtime and execution constraints

[IMPLEMENTED] Bounded embedded execution

`python_src/main.py` applies a 5-second best-effort deadline using tracing on all platforms and signals where available. It caps requests at 128 KiB, caps output at 20,000 characters, serializes HTTP execution, structures errors, blocks common unsafe imports/builtins, and keeps `SystemExit` from terminating the service.

[IMPLEMENTED] Dart bridge timeout and recovery

`lib/core/engine/python_runner.dart` performs readiness polling, sends bounded JSON requests, and applies an 8-second request guard. Android validation covered successful output, syntax errors, runtime errors, an infinite loop timeout, and a successful execution after timeout recovery.

[PARTIAL] Security isolation

The current implementation explicitly remains a constrained execution environment, not a security sandbox. A learner program can still run inside the host process and the deadline is best-effort for native or C-level blocking work.

2.2 Runtime archive and build packaging

[IMPLEMENTED] Reproducible runtime archive

`tooling/package_python_runtime.py` verifies that `app/app.zip` contains the expected `main.py` payload and matches `python_src/main.py` byte-for-byte. The latest verified archive SHA-256 is `ab973bb99f857362ae0875d3a8e489d19e0a7999059ff47756df6bcabfe208bf`.

[IMPLEMENTED] Native-symbol release packaging

Removing the contradictory `keepDebugSymbols` setting allows Flutter's AAB check to find `libflutter.so.sym` and `libapp.so.sym` in `BUNDLE-METADATA` while retaining `useLegacyPackaging` for the embedded Python payload.

2.3 Curriculum, progression, and navigation

[IMPLEMENTED] Curriculum contracts

`CurriculumRepository` validates stable curriculum metadata, unique positive module order, required module and mission fields, exact-match validation, answer contracts, MCQ options, and prerequisite references before caching the asset.

[IMPLEMENTED] Progress reconciliation

`LearningProgressService` and `ProgressManager` sanitize stale completion IDs, migrate safe legacy keys, derive unlocks, XP, level, completion percentage, resume position, and module progress from one active curriculum.

[IMPLEMENTED] Achievements and safe mission entry

Achievement definitions are catalog-backed and reconciled on dashboard load. `MissionEntryScreen` resolves route data against the active curriculum and permits only known unlocked or completed missions; invalid and locked entries never open an editor.

2.4 Accessibility and responsive interface

[IMPLEMENTED] Semantic controls

Custom settings, reset, Quick Console, streak, profile, and execution controls expose labels, hints, and button semantics instead of relying only on visual `GestureDetector` or `InkWell` presentation.

[IMPLEMENTED] Narrow screens and large text

The dashboard uses compact FORGE branding below 420 px, ellipsis for the app-bar title, a wrapping overall-progress header, and widget coverage for a 360x800 surface and 2.0 text scaling.

2.5 Local quality gates

[IMPLEMENTED] Automated test coverage

The current suite covers answer validation, curriculum contracts, progress/unlock derivation, route protection, mission rendering, settings, streaks, achievements, cold start, responsive layout, and text scaling. Direct Python tests cover runtime output, errors, blocked imports, and output truncation.

3. Validation evidence snapshot

The following results are the latest recorded local validation evidence for the current workspace. They are not a substitute for a hosted CI run or a signed distribution test.

[IMPLEMENTED] Dart analyzer

Command: flutter analyze --no-pub. Result: PASS - no issues found.

[IMPLEMENTED] Flutter tests

Command: flutter test --no-pub. Result: PASS - 31 tests.

[IMPLEMENTED] Python tests

Command: python -m unittest discover -s python_src -p test_*.py. Result: PASS - 4 tests.

[IMPLEMENTED] Runtime archive

Command: python tooling/package_python_runtime.py --check. Result: PASS - archive current.

[IMPLEMENTED] Whitespace

Command: git diff --check. Result: PASS.

[IMPLEMENTED] Debug APK

Command: flutter build apk --debug --no-pub. Result: PASS.

[IMPLEMENTED] Web build

Command: flutter build web --no-pub. Result: PASS; wasm/font warnings only.

[IMPLEMENTED] Release AAB

Command: flutter build appbundle --release --no-pub. Result: PASS after native-symbol packaging fix.

[IMPLEMENTED] Gradle symbols

Command: app:stripReleaseDebugSymbols --stacktrace --no-daemon. Result: PASS.

[IMPLEMENTED] Android device

Command: adb install/relaunch/UI dump on serial 98e86dfe. Result: PASS; controls and curriculum visible.

[IMPLEMENTED] Crash check

Command: App-scoped logcat for com.example.python_forge. Result: No FATAL EXCEPTION or AndroidRuntime crash.

RELEASE CAVEAT

The AAB now contains Flutter debug-symbol metadata, but the local release artifact is unsigned because production signing properties and a keystore were not available. Serious Python's libpythonbundle.so is a ZIP payload with a .so filename, so llvm-strip reports that it is not a valid object; AGP tolerates this and the AAB build completes.

4. Remaining gaps and new work

These items should be treated as the active backlog. They are intentionally not marked complete by the current evidence.

[BLOCKED] True killable execution sandbox

Move learner execution into a separate process or external sandbox with a reliable kill/restart boundary. Add OS-level CPU, wall-clock, memory, filesystem, and network controls, authenticated private IPC, and recovery tests for worker crashes and native blocking calls.

[PARTIAL] Real integration and device qualification

Add integration_test coverage for asset loading, first execution, success/error/timeout UI, persistence after relaunch, locked routes, keyboard behavior, and next-mission navigation. Run it across target Android API levels, ARM64, emulator, low-memory, offline, and process-restart scenarios.

[BLOCKED] Production signing and distribution

Provide the four PYTHON_FORGE_UPLOAD_* secrets through a private Gradle or CI secret store, fail closed when absent, verify the signing certificate and AAB hash, and use a Play internal-testing track before public release.

[PARTIAL] CI quality and artifact provenance

The checked-in workflow runs archive verification, pub get, analyze, Flutter tests, and a debug APK build. It still needs Python tests, integration/device tests, release AAB verification, artifact upload, checksum manifests, pinned tool versions, and signed-release gates.

[PLANNED] Curriculum graph correctness

Add prerequisite cycle detection, a machine-readable curriculum manifest with stable version and content hash, migration policy, and contract tests for all 40 missions. Keep MCQ and fill-in missions non-executing and verify failed execution never awards completion.

[PARTIAL] Documentation and version alignment

Align README, pubspec, Android versionName, and release naming. Add dedicated tests for claims such as quote normalization and document the distinction between constrained execution and a secure sandbox.

[EVIDENCE GAP] Roadmap traceability

Recover or text-extract the original PDF source if possible, preserve its SHA-256, and add an evidence index mapping each roadmap requirement to source symbols, tests, CI jobs, artifacts, and build identity.

5. Development phases

The phases below are intentionally short summaries. Each phase has a practical outcome and an exit condition so roadmap status can be updated from evidence instead of intention.

[PARTIAL] Phase 0 - Evidence baseline and traceability

Recover the original roadmap text or record its extraction limitation, preserve source and artifact hashes, add docs/review-source.md and an evidence-index file, and embed revision/source identity in future PDF generation. Exit when every roadmap item has a status and an evidence reference.

[PLANNED] Phase 1 - Execution hardening

Design and implement the killable worker or external sandbox, authenticated IPC, resource controls, timeout cleanup, and worker restart. Keep current import and builtin restrictions as defense-in-depth. Exit when adversarial and recovery tests prove a hostile or stuck program cannot hold the host app.

[PARTIAL] Phase 2 - Curriculum and validation correctness

Add curriculum manifest/version/hash/migration rules, prerequisite DAG cycle checks, contract tests across all 40 missions, and explicit starter-code and quote-normalization behavior. Exit when content changes fail fast with actionable validation errors.

[PARTIAL] Phase 3 - Integration and device qualification

Add integration tests for the real Flutter-to-runtime path, persistence, navigation, timeout recovery, keyboard, accessibility, and relaunch. Run on the target device/API/ABI matrix and publish logs and screenshots. Exit when the matrix is repeatable and artifact-linked.

[PARTIAL] Phase 4 - Release and signing

Configure secret-backed signing, fail closed without secrets, build and verify a signed AAB, retain mapping files and checksums, and complete Play internal testing. Exit when certificate identity and artifact provenance are recorded.

[PARTIAL] Phase 5 - CI and evidence automation

Pin Flutter/Python/action versions, run Python plus Dart plus integration suites, build artifacts, upload reports/checksums, and fail on stale runtime archives or missing roadmap metadata. Exit when a pull request receives the same gates as a local release candidate.

[PLANNED] Phase 6 - Content and platform expansion

Only after safety and release gates: expand curriculum, add analytics or sync where explicitly required, and broaden platform support. Exit criteria should be defined per feature rather than allowing new content to bypass the contract validator.

6. Prioritized next implementation order

1. Release configuration: provide private signing secrets, verify a signed AAB, and run internal-track testing
2. Sandbox architecture spike: choose the separate process or external sandbox boundary before adding more arbitrary-code features
3. Integration/device matrix: automate real runtime startup, timeout recovery, persistence, accessibility, and relaunch checks
4. Curriculum graph and manifest: add cycle detection, content hashes, migration policy, and all-mission contract validation
5. CI/evidence automation: upload results, checksums, symbol maps, device artifacts, and roadmap traceability metadata
6. Content expansion: proceed only after the safety and release gates are green

7. File-level evidence map

These are the principal files that support the status decisions in this revision. The complete worktree also contains the related tests, CI workflow, runtime archive, and UI changes.

[IMPLEMENTED] `python_src/main.py`

Embedded service constraints, HTTP contract, output cap, import/builtin controls, and explicit non-sandbox limitation.

[IMPLEMENTED] `lib/core/engine/python_runner.dart`

Dart readiness polling, JSON request bridge, and request timeout.

[IMPLEMENTED] `tooling/package_python_runtime.py`

Deterministic app.zip creation and freshness check.

[IMPLEMENTED] `lib/features/curriculum/data/repositories/curriculum_repository.dart`

Curriculum parsing and validation contract.

[IMPLEMENTED] `lib/core/progression/learning_progress.dart`

Derived unlocks, prerequisites, XP, level, resume, and stale-ID sanitization.

[IMPLEMENTED] `lib/core/progression/progress_manager.dart`

Idempotent completion, safe legacy migration, and progress-only reset.

[IMPLEMENTED] `lib/features/curriculum/presentation/screens/mission_screen.dart`

Mission validation, execution, completion, streak, achievements, and next mission.

[IMPLEMENTED] `lib/features/curriculum/presentation/screens/home_screen.dart`

Dashboard startup reconciliation, responsive title/progress layout, and semantic controls.

[IMPLEMENTED] `.github/workflows/flutter.yml`

Current local CI definition and its remaining debug-only scope.

[IMPLEMENTED] `android/app/build.gradle.kts`

Native packaging and conditional production signing configuration.

[IMPLEMENTED] `test/` and `python_src/test_main.py`

Curriculum, answer, progress, navigation, UI, settings, and direct runtime tests.

8. Risks, assumptions, and decisions

- The embedded runtime is bounded constrained execution, not a secure sandbox. Do not make a stronger security claim until Phase 1 is complete.
- A release AAB build passing is not equivalent to a distributable release; signing and certificate verification are still required.
- One Android device smoke pass is useful evidence but is not a device compatibility matrix.
- The original PDF remains unchanged. This updated PDF is a new artifact and does not assert that unreadable original body text was reproduced exactly.
- No commit or push is implied by this document. The repository owner should approve the release and sandbox decisions before creating a checkpoint commit.

9. Revision change log

- Added explicit status labels for implemented, partial, planned, blocked, and evidence-gap work.
- Recorded completed runtime, packaging, curriculum, progression, navigation, accessibility, build, test, and device changes.
- Added the current validation command/result snapshot and release-symbol packaging explanation.
- Added remaining blockers and new work for sandbox isolation, integration/device qualification, signing, CI, curriculum DAG checks, version alignment, and evidence traceability.
- Added brief Phases 0 through 6 and a prioritized next implementation order.

- Preserved Python_Forge_Comprehensive_Review_and_Roadmap.pdf without overwriting it.

CURRENT DECISION

Python Forge is ready for continued internal development and controlled testing. It is not yet ready to claim secure arbitrary-code execution or public production distribution. The next highest-value work is the sandbox architecture decision followed by signed-release and integration/device gates.